

Week 6 Lab: RTOS Foundations with FreeRTOS

Overview

Create a multi-task FreeRTOS application with concurrent LED blinking and button input handling. This lab demonstrates how an RTOS enables multiple independent tasks to run simultaneously with deterministic timing.

Time allocation: 90 minutes

- CubeMX project setup: 20 min
- LED blink tasks implementation: 25 min
- Button task with priority: 20 min
- Debugging and observation: 15 min
- Documentation and cleanup: 10 min

Learning Objectives

By the end of this lab, you will be able to:

1. Create a FreeRTOS project using STM32CubeMX
2. Configure multiple tasks with different priorities
3. Use `osDelay()` for non-blocking timing in tasks
4. Implement button input handling in a separate task
5. Observe task execution using the debugger's RTOS views
6. Identify and fix common RTOS mistakes (`HAL_Delay`, stack issues)

Safety Gate

Safety Checklist (Low-Risk Lab)

This lab uses only LEDs and buttons - low voltage, minimal risk.

- ☐ USB cable connected to Nucleo board
- ☐ External LEDs have current-limiting resistors (330Ω)
- ☐ Button has pull-up/pull-down resistor if external (or use onboard B1)
- ☐ Bench supply OFF (not needed - USB power is sufficient)

Good Habits

Even for low-risk labs:

- Verify connections before programming
- GPIO total current $< 100\text{mA}$ (plenty for 3 LEDs)
- Never short GPIO pins to VCC or GND directly

Required Parts and Tools

Hardware

- STM32 Nucleo-F401RE board (with onboard LED on PA5, button on PC13)
- External LEDs ($\times 2$) - different colors recommended
- Resistors: 330Ω ($\times 2$) for LED current limiting
- Breadboard and jumper wires

Software

- STM32CubeIDE (latest version)
- STM32CubeMX (integrated in CubeIDE)

! PRE-LAB SECTION

Complete the following BEFORE arriving at the lab session.

Pre-Lab Preparation

1. Understand Task Timing

Given the following task configuration, calculate when each LED toggles:

Task	Delay (ms)	LED State After
FastLED	100	Toggle every 100ms = _____ Hz blink rate
SlowLED	500	Toggle every 500ms = _____ Hz blink rate
Heartbeat	1000	Toggle every 1000ms = _____ Hz blink rate

Note: If LED toggles every T ms, one complete ON-OFF cycle = 2T ms

2. Priority Prediction

With these task priorities, predict behavior:

Task	Priority	What happens if this task never blocks?
ButtonTask	osPriorityHigh	_____
FastLED	osPriorityNormal	_____
SlowLED	osPriorityNormal	_____
Heartbeat	osPriorityLow	_____

Question: If ButtonTask runs an infinite loop without osDelay(), which other tasks will run?

Answer: _____

3. Stack Size Calculation

Each task needs stack space. For simple GPIO operations:

Item	Typical Size (words)
Local variables	4-8
Function call overhead	8-16
Context save area	16-32
Safety margin	32
Total recommended	128 words

128 words $\times$ 4 bytes/word = _____ bytes per task

4 tasks $\times$ _____ bytes = _____ bytes total stack

i IN-LAB SECTION

The following sections are completed during the lab session with TA supervision.

Part 1: CubeMX Project Setup (20 minutes)

Step 1.1: Create New Project

1. Open STM32CubeIDE
2. File → New → STM32 Project
3. Select board: NUCLEO-F401RE
4. Project name: W06_RTOS_Lab
5. Click Finish

Step 1.2: Configure GPIO Pins

In CubeMX Pinout view:

Pin	Mode	User Label
PA5	GPIO_Output	LED_GREEN (onboard)
PA0	GPIO_Output	LED_FAST
PA1	GPIO_Output	LED_SLOW
PC13	GPIO_Input	BTN_USER (onboard)

For PA0 and PA1:

- GPIO mode: Output Push-Pull
- GPIO output level: Low
- Maximum output speed: Low
- No pull-up/pull-down

For PC13 (already configured if using onboard button):

- GPIO mode: Input
- GPIO Pull: Pull-up (button connects to GND)

Step 1.3: Enable FreeRTOS

1. Go to Middleware → FREERTOS

2. Mode: **CMSIS_V2**
3. Click Configuration

In Config parameters tab:

Parameter	Value	Notes
TOTAL_HEAP_SIZE	15360	Default is fine
TICK_RATE_HZ	1000	1ms tick
MAX_PRIORITIES	7	Default
MINIMAL_STACK_SIZE	128	Words
USE_PREEMPTION	Enabled	Required

Enable hooks (for debugging):

Hook	Enable?
USE_IDLE_HOOK	Yes
USE_TICK_HOOK	No
CHECK_FOR_STACK_OVERFLOW	Option 2

Step 1.4: Create Tasks in CubeMX

In Tasks tab, add 4 tasks:

Task 1: ButtonTask

Field	Value
Task Name	ButtonTask
Priority	osPriorityHigh
Stack Size (words)	128
Entry Function	StartButtonTask
Code Generation Option	Default
Parameter	NULL
Allocation	Dynamic

Task 2: FastLEDTask

Field	Value
Task Name	FastLEDTask
Priority	osPriorityNormal
Stack Size (words)	128
Entry Function	StartFastLEDTask
Code Generation Option	Default

Task 3: SlowLEDTask

Field	Value
Task Name	SlowLEDTask
Priority	osPriorityNormal
Stack Size (words)	128
Entry Function	StartSlowLEDTask
Code Generation Option	Default

Task 4: HeartbeatTask

Field	Value
Task Name	HeartbeatTask
Priority	osPriorityLow
Stack Size (words)	128
Entry Function	StartHeartbeatTask

Field	Value
Code Generation Option	Default

Step 1.5: Generate Code

1. Save CubeMX configuration (Ctrl+S)
2. Click “Generate Code” button
3. Open project in STM32CubeIDE when prompted

Verify: Check that `freertos.c` contains 4 task function stubs.

Part 2: Implement LED Tasks (25 minutes)

Step 2.1: Locate Task Functions

Open Core/Src/freertos.c. Find the generated task functions:

```
void StartButtonTask(void *argument)
{
    for(;;)
    {
        osDelay(1);
    }
}

void StartFastLEDTask(void *argument)
{
    for(;;)
    {
        osDelay(1);
    }
}
// ... etc
```

Step 2.2: Implement HeartbeatTask

Replace the HeartbeatTask function:

```
void StartHeartbeatTask(void *argument)
{
    /* Heartbeat: Toggle LED every 1 second
     * Uses onboard green LED (PA5)
     * Lowest priority - runs when nothing else needs CPU
     */
    for(;;)
    {
        HAL_GPIO_TogglePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin);
        osDelay(1000); // 1000ms = 1 second
    }
}
```

Step 2.3: Implement FastLEDTask

```
void StartFastLEDTask(void *argument)
{
    /* Fast blink: Toggle LED every 100ms
     * Normal priority - standard operation
     */
    for(;;)
    {
        HAL_GPIO_TogglePin(LED_FAST_GPIO_Port, LED_FAST_Pin);
        osDelay(100); // 100ms toggle = 5Hz blink rate
    }
}
```

Step 2.4: Implement SlowLEDTask

```
void StartSlowLEDTask(void *argument)
{
    /* Slow blink: Toggle LED every 500ms
     * Normal priority - runs alongside FastLED
     */
    for(;;)
    {
        HAL_GPIO_TogglePin(LED_SLOW_GPIO_Port, LED_SLOW_Pin);
        osDelay(500); // 500ms toggle = 1Hz blink rate
    }
}
```

Step 2.5: Build and Test LED Tasks

1. Build project (Ctrl+B)
2. Program board (Run → Debug or F11)
3. Resume execution (F8)

Verify all 3 LEDs blink at different rates:

LED	Expected Behavior
Green (PA5)	Slow 0.5Hz blink (1s on, 1s off)
LED_FAST (PA0)	Fast 5Hz blink
LED_SLOW (PA1)	Medium 1Hz blink

LED	Expected Behavior
-----	-------------------

Checkpoint: TA verifies LEDs blinking independently:

Part 3: Button Task Implementation (20 minutes)

Step 3.1: Add Shared Variable

At the top of `freertos.c` (in USER CODE area):

```
/* USER CODE BEGIN Variables */
volatile uint8_t fast_mode = 0; // 0 = normal (100ms), 1 = fast (50ms)
/* USER CODE END Variables */
```

Step 3.2: Implement ButtonTask

```
void StartButtonTask(void *argument)
{
    /* Button handler: Detect press and toggle mode
     * High priority - responds immediately to user input
     * Uses simple polling with debounce
     */
    GPIO_PinState last_state = GPIO_PIN_SET; // Button not pressed (pull-up)
    GPIO_PinState current_state;

    for(;;)
    {
        current_state = HAL_GPIO_ReadPin(BTN_USER_GPIO_Port, BTN_USER_Pin);

        // Detect falling edge (button pressed - active low)
        if (last_state == GPIO_PIN_SET && current_state == GPIO_PIN_RESET)
        {
            // Simple debounce: wait and verify
            osDelay(50); // 50ms debounce delay
            if (HAL_GPIO_ReadPin(BTN_USER_GPIO_Port, BTN_USER_Pin) == GPIO_PIN_RESET)
            {
                // Toggle mode
                fast_mode = !fast_mode;
            }
        }

        last_state = current_state;
        osDelay(10); // Poll every 10ms
    }
}
```

Step 3.3: Modify FastLEDTask to Use Mode

Update FastLEDTask to respond to button:

```
void StartFastLEDTask(void *argument)
{
    /* Fast blink with variable rate
     * Rate controlled by button (fast_mode variable)
     */
    uint32_t delay_time;

    for(;;)
    {
        HAL_GPIO_TogglePin(LED_FAST_GPIO_Port, LED_FAST_Pin);

        // Select delay based on mode
        delay_time = fast_mode ? 50 : 100; // 50ms or 100ms
        osDelay(delay_time);
    }
}
```

Note: Accessing `fast_mode` without mutex is safe here because:

- Only one task writes (ButtonTask)
- The value is a single byte (atomic read)
- Timing variation is acceptable

Step 3.4: Test Button Functionality

1. Rebuild and program (F11)
2. Observe LED_FAST blinking at 100ms rate
3. Press user button (B1)
4. Observe LED_FAST change to 50ms rate
5. Press again - should toggle back

Test	Result
Initial fast LED rate	100ms / 5Hz:
After button press	50ms / 10Hz:
After second press	100ms / 5Hz:

Checkpoint: TA verifies button changes LED rate:

Part 4: RTOS Debugging and Observation (15 minutes)

Step 4.1: Enable RTOS Debugging

1. Start debugging session (F11)
2. Open Window → Show View → FreeRTOS → Task List

If Task List view unavailable: Window → Show View → Other → FreeRTOS

Step 4.2: Observe Task States

Pause execution (Suspend button) and observe:

Task	State	Stack High Water Mark
ButtonTask	_____	_____ words unused
FastLEDTask	_____	_____ words unused
SlowLEDTask	_____	_____ words unused
HeartbeatTask	_____	_____ words unused
IDLE	_____	_____ words unused

High Water Mark: Minimum free stack space ever (lower = closer to overflow)

Question: Why are most tasks in “Blocked” state?

Answer: _____

Step 4.3: Demonstrate HAL_Delay Problem

TEMPORARILY modify HeartbeatTask to use HAL_Delay:

```
void StartHeartbeatTask(void *argument)
{
    for(;;)
    {
        HAL_GPIO_TogglePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin);
        HAL_Delay(1000); // WRONG! Uses busy-wait
    }
}
```

1. Rebuild and run
2. **Observe:** What happens to other LEDs?

Observation	Result
Do other LEDs still blink?	YES / NO
Is the system responsive?	YES / NO

Explanation: _____

REVERT the change back to `osDelay(1000)` before continuing!

Step 4.4: Priority Experiment

Temporarily change HeartbeatTask priority to `osPriorityRealtime` (highest):

In CubeMX: Tasks tab → HeartbeatTask → Priority → `osPriorityRealtime`

Regenerate code and test.

Observation	Result
Does system still work?	YES / NO
Any visible difference?	_____

Why does it still work? _____

Restore priority to `osPriorityLow` before proceeding.

Step 4.5: Stack Overflow Demonstration (Optional)

If time permits, create intentional stack overflow:

1. Create new task with stack size = 32 words (too small)
2. Add large local array in task function
3. Observe crash or stack overflow hook trigger

```
// Example of stack-hungry task (DON'T use in real code)
void StartBadTask(void *argument)
{
    char big_buffer[200]; // Uses too much stack!
    for(;;)
    {
        // ...
    }
}
```

Implement the stack overflow hook in freertos.c:

```
/* USER CODE BEGIN Application */
void vApplicationStackOverflowHook(TaskHandle_t xTask,
                                   char *pcTaskName)
{
    // Breakpoint here, or blink LED rapidly
    __BKPT(0); // Halt debugger
}
/* USER CODE END Application */
```

Observation: Did the hook get called? YES / NO

Measurement Summary

ID	Measurement	Expected	Measured	Pass?
M1	FastLED period (normal)	200ms (5Hz)		
M2	FastLED period (fast mode)	100ms (10Hz)		
M3	SlowLED period	1000ms (1Hz)		
M4	Heartbeat period	2000ms (0.5Hz)		
M5	Button response time	<50ms		
M6	Tasks in Blocked state	4 of 4 (most time)		

Note: LED toggle period = half of blink cycle period (on-off-on)

Failure Injection Analysis

Complete this table by predicting and then testing:

Scenario	Prediction	Actual Result
HAL_Delay instead of osDelay		
Remove osDelay from ButtonTask		
All tasks same priority		
Stack size = 32 words		
osDelay(0) instead of osDelay(10)		

Analysis Questions

Answer in your lab report (2-3 sentences each):

1. Why do all LED tasks appear to run simultaneously even though there's only one CPU core?

2. What is the advantage of using osDelay() over HAL_Delay() in RTOS tasks?

3. Why is the ButtonTask given higher priority than the LED tasks?

4. If you wanted to add another task that must run exactly every 10ms, what considerations would you have?

Deliverables Checklist

Before leaving the lab, ensure you have:

- ☐ Working project with 4 tasks running concurrently
- ☐ Three LEDs blinking at different rates independently
- ☐ Button press changes FastLED blink rate
- ☐ HAL_Delay vs osDelay behavior documented
- ☐ Task List debugger view screenshot
- ☐ Completed measurement summary table
- ☐ Answered analysis questions

Grading Rubric

Criterion	Points	Description
FreeRTOS setup	20	CubeMX configuration correct, project builds
LED tasks working	25	All 3 LEDs blink independently at correct rates
Button task	25	Mode toggle works, proper debounce
Debugging observations	20	Task states observed, HAL_Delay problem demonstrated
Analysis & documentation	10	Thoughtful answers, complete measurements

Total: 100 points

Common Issues and Solutions

Issue	Symptom	Solution
No tasks run	LEDs don't blink	Check osKernelStart() is called
Only one LED blinks	Others stuck	Check for HAL_Delay, infinite loops
Random crashes	Unpredictable behavior	Increase stack sizes
Button unresponsive	No mode change	Check GPIO configuration, button wiring

Issue	Symptom	Solution
Build errors	Compilation fails	Check task function names match CubeMX
Task List empty	No tasks shown	Enable FreeRTOS debug in CubeMX

Cleanup Checklist

Before leaving:

- ☐ Stop debug session
- ☐ Save project files
- ☐ Export or push project to version control
- ☐ Disconnect USB cable
- ☐ Return oscilloscope probes (if used)
- ☐ Tidy breadboard

Key Takeaways

1. **FreeRTOS enables concurrent execution** - Multiple tasks share CPU time
2. **Use `osDelay()`, not `HAL_Delay()`** - Blocking delays prevent other tasks from running
3. **Priority determines execution order** - Higher priority tasks preempt lower ones
4. **Tasks should spend most time blocked** - Waiting for delays, events, or resources
5. **Stack sizing matters** - Too small causes crashes, too large wastes RAM